

자료구조 및 알고리즘

Chapter 7. 큐

7.1 큐란

7.2 리스트를 이용한 큐

7.3 연결 리스트를 이용한 큐

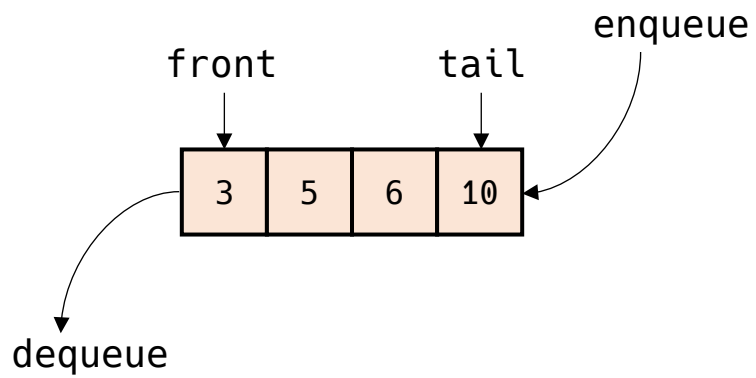
7.4 큐 응용

1. 큐란

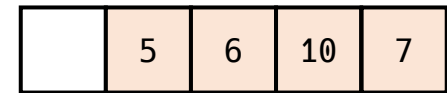
- 큐란?
 - 선입선출로 데이터를 넣고 빼는 자료구조



▪ 큐의 개념과 원리



dequeue



enqueue(7)

- 큐의 ADT

맨 끝에 원소를 추가한다

맨 앞의 원소를 삭제하면서 알려준다

맨 앞의 원소를 알려준다

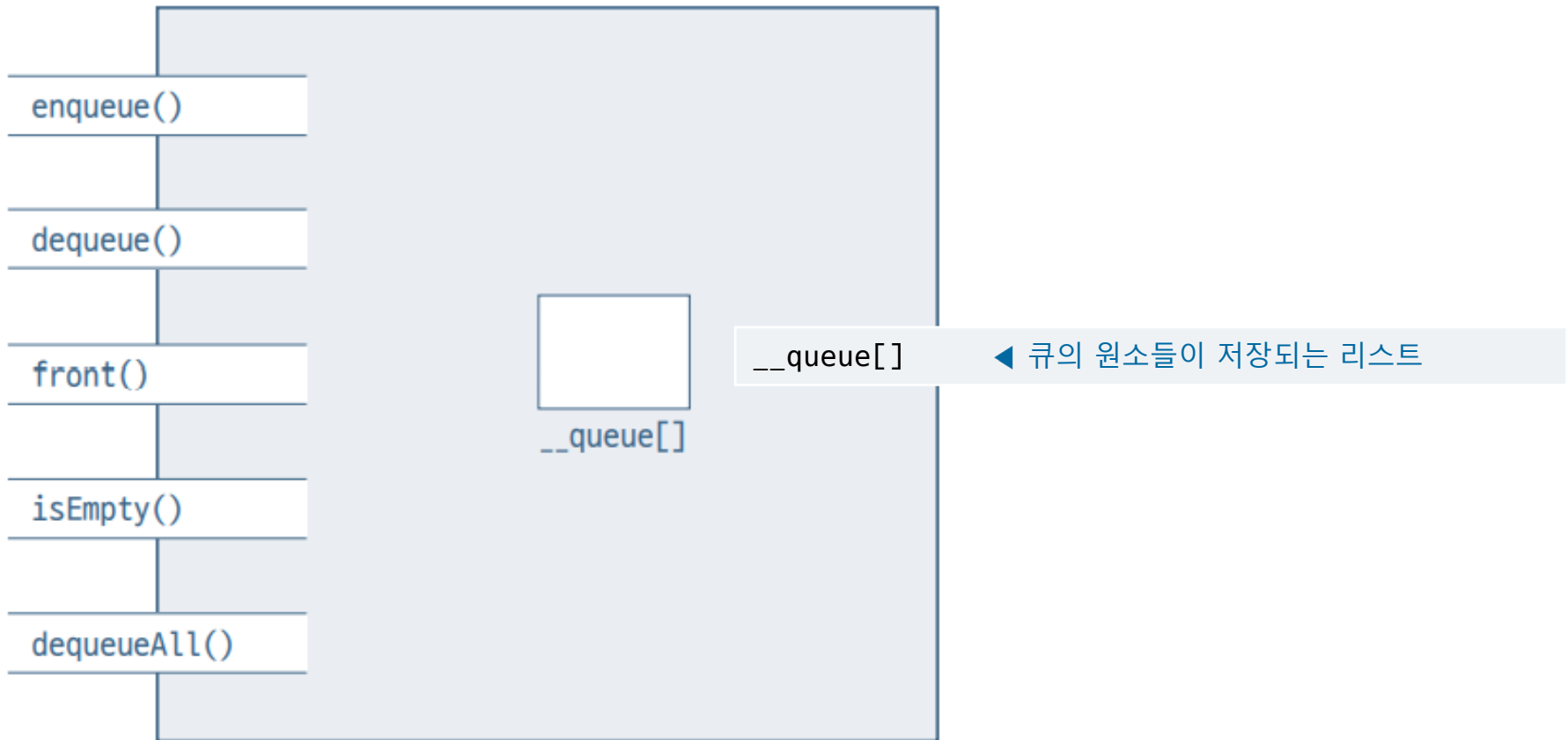
큐가 비어 있는지 확인한다

큐를 깨끗이 비운다

그림 7-3 ADT 큐

2. 리스트를 이용한 큐

- 리스트 큐의 객체 구조



▪ 리스트 큐의 객체 구조

enqueue()	enqueue(x) ◀ 큐의 끝에 원소 x를 삽입한다.
dequeue()	dequeue() ◀ 큐의 맨 앞에 있는 원소를 알려주고 삭제한다.
front()	front() ◀ 큐의 맨 앞에 있는 원소를 알려준다.
isEmpty()	isEmpty() ◀ 큐가 비어 있는지 알려준다.
dequeueAll()	dequeueAll() ◀ 큐를 깨끗이 청소한다.

- 리스트 큐의 작업과 구현

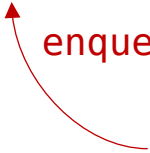
- listQueue.py

```
class ListQueue:
    def __init__(self):
        self.__queue = []
    def enqueue(self, x): ...
    def dequeue(self): ...
    def front(self): ...
    def isEmpty(self): ...
```

- 리스트 큐의 작업과 구현
 - 원소 삽입

`--queue[]` `["Kruth", "Smith", 30, "Bush", "Kane"]`

`enqueue(50)`
50



- 리스트 큐의 작업과 구현

- 원소 삽입 ~~삽입~~ ~~추가~~.

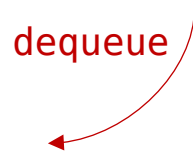
`__queue[]` `["Kruth", "Smith", 30, "Bush", "Kane", 50]`

```
def enqueue(self, x):  
    self.__queue.append(x)
```

- 리스트 큐의 작업과 구현
 - 원소 삽입

`--queue[]` `["Kruth", "Smith", 30, "Bush", "Kane"]`

dequeue



- 리스트 큐의 작업과 구현
 - 원소 삭제

`__queue[]`

`["Smith", 30, "Bush", "Kane"]`

```
def dequeue(self):  
    self.__queue.pop(0)
```

- 리스트 큐의 기타 작업

```
def front(self):  
    return self.__queue[0]
```

self.__queue.getNode(0)

```
def isEmpty(self) -> bool:  
    return (len(self.__queue) == 0)
```

self.__queue.isEmpty()

```
def dequeueAll(self):  
    self.__queue = []
```

Circular linked list()

- 구조적 문제

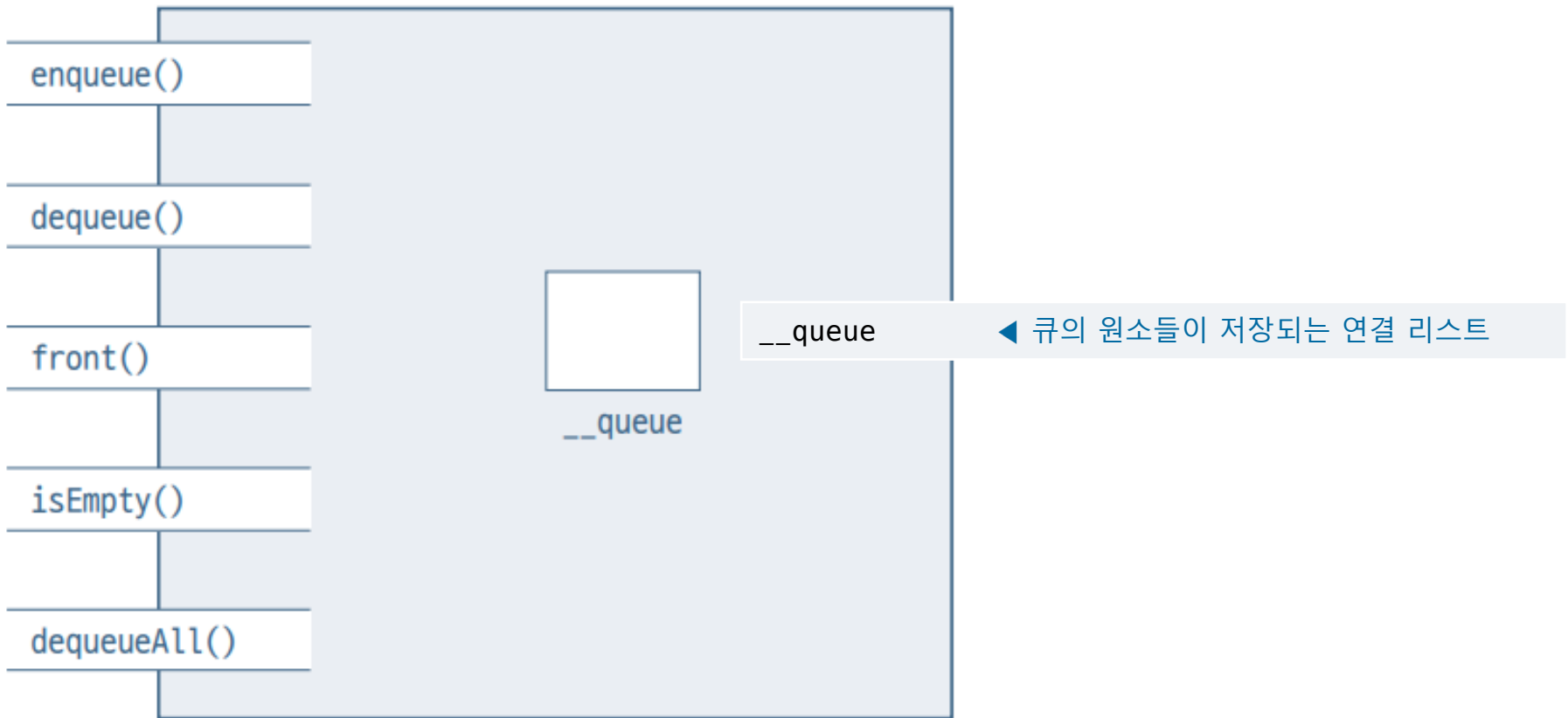
나) 원형차.

- 단순 배열 리스트로 구현하면 공간이 있는데도 큐가 꽉 찬 것처럼 보임

대형 S1AM

3. 연결 리스트를 이용한 큐

- 연결 리스트 큐의 객체 구조



■ 연결 리스트 큐의 객체 구조

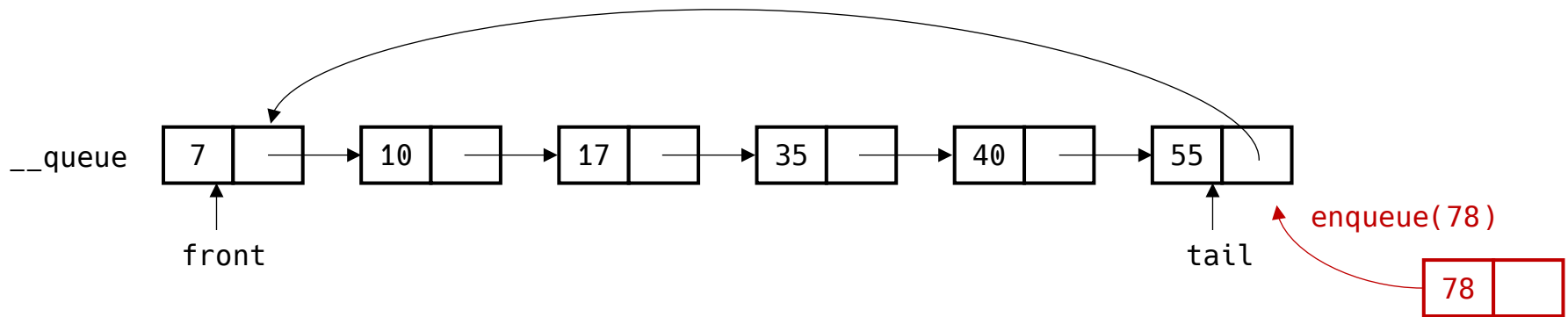
enqueue()	enqueue(x) ◀ 큐의 끝에 원소 x를 삽입한다.
dequeue()	dequeue() ◀ 큐의 맨 앞에 있는 원소를 알려주고 삭제한다.
front()	front() ◀ 큐의 맨 앞에 있는 원소를 알려준다.
isEmpty()	isEmpty() ◀ 큐가 비어 있는지 알려준다.
dequeueAll()	dequeueAll() ◀ 큐를 깨끗이 청소한다.

- 연결 리스트 큐의 작업과 구현

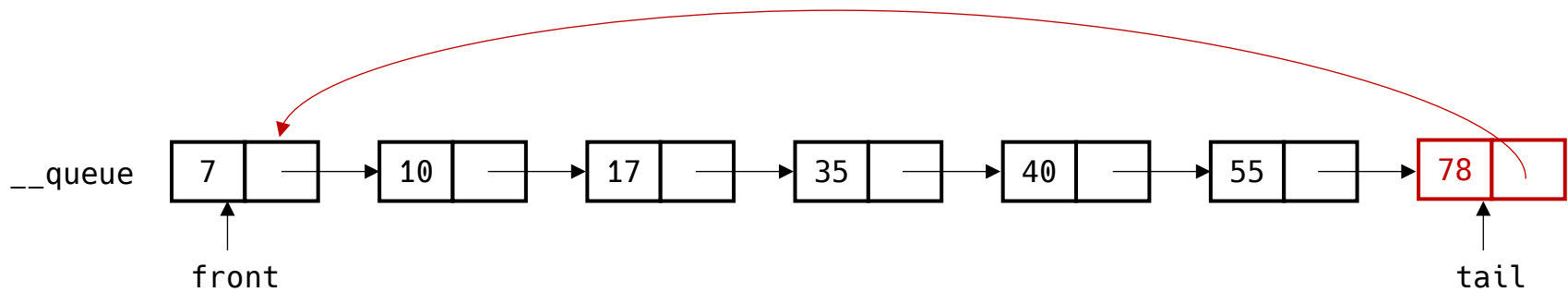
- linkedQueue.py

```
class LinkedQueue:
    def __init__(self):
        self.__queue = CircularLinkedList()
    def enqueue(self, x): ...
    def dequeue(self): ...
    def front(self): ...
    def isEmpty(self): ...
```

- 연결 리스트 ~~스택~~의 작업과 구현
 - 원소 삽입

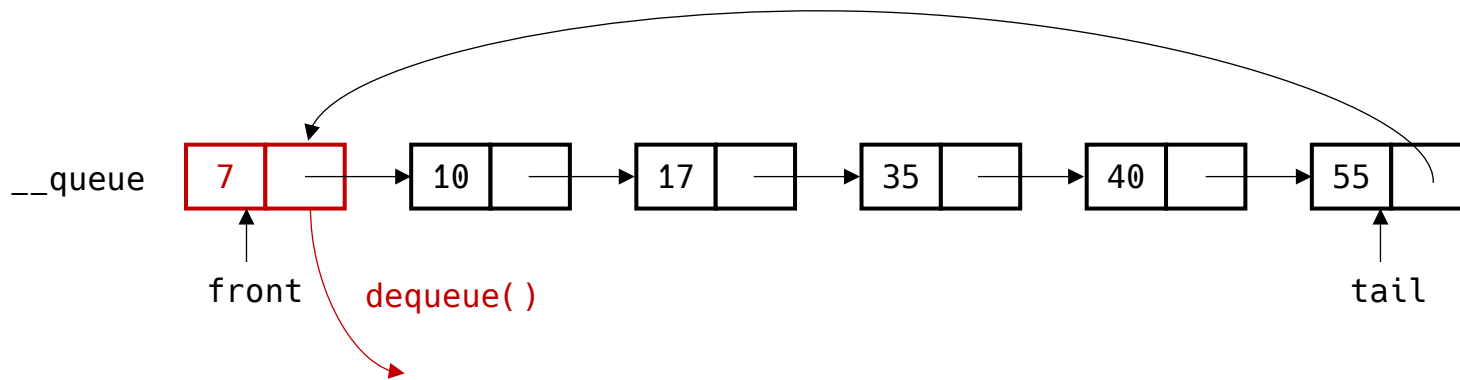


- 연결 리스트 스택의 작업과 구현
 - 원소 삽입

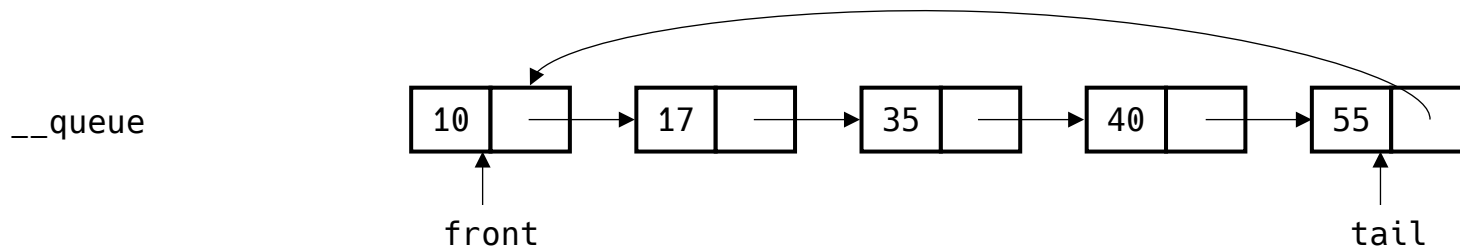


```
def enqueue(self):  
    self.__queue.append(x)
```

- 연결 리스트 스택의 작업과 구현
 - 원소 삭제



- 연결 리스트 ~~스택~~의 작업과 구현
 - 원소 삭제



```
def dequeue(self):  
    return self.__queue.pop(0)
```

4. 큐 응용

▪ 좌우동형 문자열 체크

→ stack, queue
2개 다 사용

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

Palindrome Check!

큐 앞에 값 체크 → 비교.
스택 뒤에 값 체크 →

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

Palindrome Check!

str = 'lioninoil'

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

Palindrome Check!

str = 'lioninoil'

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

q

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

l	i	o	n	i	n	o	i	l
---	---	---	---	---	---	---	---	---

q

l	i	o	n	i	n	o	i	l
---	---	---	---	---	---	---	---	---

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

l	i	o	n	i	n	o	i	l
---	---	---	---	---	---	---	---	---

q

l	i	o	n	i	n	o	i	l
---	---	---	---	---	---	---	---	---

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

l	i	o	n	i	n	o	i
---	---	---	---	---	---	---	---

q

i	o	n	i	n	o	i	l
---	---	---	---	---	---	---	---

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

l	i	o	n	i	n	o
---	---	---	---	---	---	---

q

o	n	i	n	o	i	l
---	---	---	---	---	---	---

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

l	i	o	n	i	n
---	---	---	---	---	---

q

n	i	n	o	i	l
---	---	---	---	---	---

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

l	i	o	n	i
---	---	---	---	---

q

i	n	o	i	l
---	---	---	---	---

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

l	i	o	n
---	---	---	---

q

n	o	i	l
---	---	---	---

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

l	i	o
---	---	---

q

o	i	l
---	---	---

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

l	i
---	---

q

i	l
---	---

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s l

q l

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

q

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

q

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

A = 'lioninoil'

s

q

▪ 좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

Palindrome Check!

str = 'lioninoil'

t = True

lioninoil is Palindrome?: True